

Bloque 4 — Diseño y construcción de pipeline ETL/ELT

Módulo: Extracción, Transformación y Carga de Datos desde Fuentes Múltiples (5104) **Unidad Didáctica:** UD1 — Tipología y características de las fuentes de datos **Duración estimada:** ~6 h **Curso académico:** 2026-2027

Contenidos de este bloque

Epígrafe	Contenido
4.1	ETL vs ELT: arquitecturas, diferencias y cuándo usar cada una
4.2	Construcción de pipelines ETL con Python puro
4.3	Transformaciones: limpieza, normalización y feature engineering
4.4	Orquestación de pipelines: Airflow y alternativas
4.5	Calidad de datos y monitoreo en producción
4.6	Pipeline completo: de fuentes múltiples a dataset ML-ready

Criterios de evaluación asociados

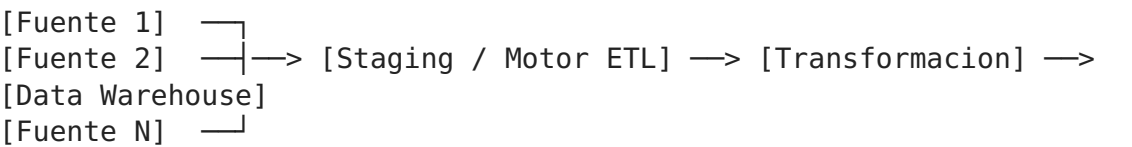
(CE i) Se han diseñado y construido pipelines de extracción, transformación y carga de datos adaptados a distintas fuentes y formatos. (CE j) Se han aplicado técnicas de validación y control de calidad de datos dentro del pipeline.

Librerías cargadas correctamente.

4.1 ETL vs ELT: arquitecturas y cuándo usar cada una

ETL — Extract, Transform, Load

El modelo clásico: los datos se extraen de las fuentes, se transforman en un servidor intermedio (menudo llamado *staging area*) y finalmente se cargan en el destino ya limpios.



Ventajas:

- El destino recibe datos ya limpios y listos para consultar.
- Protege el destino de datos sucios.

- Ideal cuando el destino tiene capacidad de cómputo limitada.

ELT — Extract, Load, Transform

El modelo moderno: los datos se cargan **en bruto** en el destino (normalmente un data lake o un warehouse en la nube) y la transformación se realiza **dentro** del propio destino, aprovechando su potencia de cómputo.

```

[Fuente 1]  ┌
[Fuente 2] ─┤─> [Data Lake / DW en nube] ─> [Transformación in-
[Fuente N]  └           situ] ─> [Tablas analíticas]
  
```

Ventajas:

- El dato en bruto siempre queda disponible (trazabilidad).
- Los data warehouses modernos (BigQuery, Snowflake, Redshift) son muy potentes para transformar.
- Fácil re-transformar sin re-extraer.

¿Cuándo usar cada uno?

Criterio	ETL	ELT
Datos sensibles (GDPR)	Mejor: anonimizar antes de cargar	Con cuidado
Volumen muy grande	Puede saturar el staging	Ideal
Destino legacy (capacidad limitada)	Ideal	No recomendado
Agilidad / re-transformar	Lento (re-extraer)	Rápido
Herramientas	Informatica, Talend, dbt+Airflow	dbt, Spark SQL, BigQuery

El diagrama siguiente compara ambas arquitecturas.

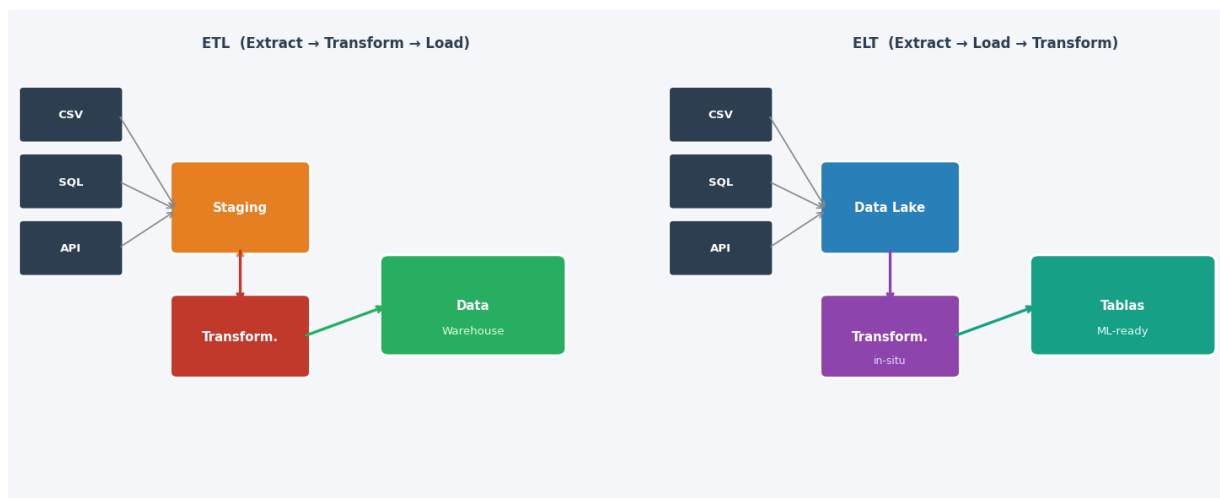


Diagrama 1: ETL vs ELT

4.2 Construcción de pipelines ETL con Python puro

Un pipeline ETL bien diseñado sigue el principio de **separación de responsabilidades**: cada etapa hace una sola cosa y las etapas se encadenan limpiamente.

Patrón de diseño: funciones encadenables

```
def extraer(ruta: str) -> pd.DataFrame: ...
def transformar(df: pd.DataFrame) -> pd.DataFrame: ...
def cargar(df: pd.DataFrame, destino: str): ...

# Pipeline
df_raw = extraer('fuente.csv')
df_limpio = transformar(df_raw)
cargar(df_limpio, 'destino.db')
```

Diseño robusto: contexto de ejecución

Un pipeline de producción registra metadatos de cada ejecución: cuántos registros entraron, cuántos salieron, cuánto tardó, qué errores hubo.

Vamos a construir un pipeline completo con tres fuentes heterogéneas.

El problema: datos industriales dispersos en tres sistemas

Imaginemos una planta de fabricación que monitoriza cuatro sensores físicos (S01–S04). Los datos relevantes para entrenar un modelo de **detección de anomalías** están repartidos en tres sistemas distintos, con formatos distintos y calidad distinta:

Fuente 1 — CSV de lecturas de sensores

Cada fila es una lectura tomada cada 10 minutos por uno de los cuatro sensores. Contiene temperatura, presión y si el sensor estaba activo en ese momento. Este fichero tiene problemas de calidad heredados de los sistemas SCADA:

- Algunos valores de temperatura son `NaN` (fallo de comunicación con el sensor).
- Unos pocos valores son `999.0`, un código de error del sensor que debe tratarse como nulo.
- La columna `activo` puede valer `True`, `False` o `None`.

Transformaciones necesarias: convertir tipos, eliminar outliers físicamente imposibles (temperatura > 200 °C), imputar nulos con la mediana del propio sensor, extraer características temporales (hora de la semana, turno laboral) y normalizar.

Fuente 2 — SQLite de averías

Cada fila registra una avería ocurrida en un sensor: fecha, sensor afectado y gravedad (`baja`, `alta`). Esta tabla la mantiene el equipo de mantenimiento.

Transformaciones necesarias: convertir la fecha a tipo `datetime`, codificar la gravedad con número ordinal (1/2/3) y agregar por sensor para obtener el **número total de averías** y la **grave máxima histórica** de cada sensor.

Fuente 3 — JSON de metadatos de sensores

Un fichero JSON con información estática de cada sensor: en qué planta está instalado (A o B), fabricante (Siemens, ABB, Honeywell) y la fecha de instalación.

Transformaciones necesarias: pivotar el JSON a formato tabular (un sensor por fila) y codifica variables categóricas para que el modelo pueda usarlas.

Resultado esperado

Al final del pipeline tendremos un único `DataFrame` con una fila por lectura de sensor, que co
señal temporal (temperatura, presión, turno), el historial de averías del sensor y sus metadatos f
Este dataset estará listo para ser usado directamente como entrada a un clasificador binario (lec
normal vs. anomalía).

Grupo de columnas	Ejemplos
Señal temporal	<code>temperatura</code> , <code>presion</code> , <code>temp_norm</code> , <code>temp_media_6h</code> , <code>turr</code>
Historial de averías	<code>n_averias</code> , <code>max_gravedad</code>
Metadatos del sensor	<code>planta</code> , <code>fabricante</code>
Variable objetivo	<code>anomalia</code> (1 si <code>temperatura > media_sensor + 2·σ</code>)

```
In [3]: # — Generar tres fuentes de datos sintéticas —————
np.random.seed(0)

# Fuente 1: CSV de sensores industriales
n1 = 1000
df_sensores = pd.DataFrame({
    'timestamp': pd.date_range('2024-01-01', periods=n1, freq='10min').as
    'sensor_id': np.random.choice(['S01', 'S02', 'S03', 'S04'], n1),
    'temperatura': np.round(np.random.normal(65, 12, n1), 2),
    'presion': np.round(np.random.normal(2.5, 0.4, n1), 3),
    'activo': np.random.choice([True, False, None], n1, p=[0.8, 0.15
})
# Introducir valores faltantes y outliers a proposito
df_sensores.loc[np.random.choice(n1, 40, replace=False), 'temperatura'] =
df_sensores.loc[np.random.choice(n1, 5, replace=False), 'temperatura'] =

RUTA_CSV = Path('/tmp/sensores.csv')
df_sensores.to_csv(RUTA_CSV, index=False)

# Fuente 2: SQLite con tabla de averias
conn = sqlite3.connect('/tmp/averias.db')
n2 = 200
pd.DataFrame({
    'id': range(1, n2 + 1),
    'sensor_id': np.random.choice(['S01', 'S02', 'S03', 'S04', 'S05'], n2
```

```

    'gravedad': np.random.choice(['baja', 'media', 'alta'], n2, p=[0.6,
    'fecha': [(datetime(2024, 1, 1) + timedelta(days=int(d))).strftime(
        for d in np.random.randint(0, 180, n2)],
    })).to_sql('averias', conn, if_exists='replace', index=False)
conn.close()

# Fuente 3: JSON con metadatos de sensores
metadatos = {s: {'planta': np.random.choice(['A', 'B']),
    'fabricante': np.random.choice(['Siemens', 'ABB', 'Honey
    'instalado': f'202{np.random.randint(0, 4)}-0{np.random
        for s in ['S01', 'S02', 'S03', 'S04']}]
Path('/tmp/metadatos.json').write_text(json.dumps(metadatos, indent=2))

print(f"Fuente 1 (CSV): {len(df_sensores):,} filas -> {RUTA_CSV}")
print(f"Fuente 2 (SQLite): {n2} averias -> /tmp/averias.db")
print(f"Fuente 3 (JSON): {len(metadatos)} sensores -> /tmp/metadatos.

```

```

Fuente 1 (CSV): 1,000 filas -> /tmp/sensores.csv
Fuente 2 (SQLite): 200 averias -> /tmp/averias.db
Fuente 3 (JSON): 4 sensores -> /tmp/metadatos.json

```

```

In [4]: # — Etapa EXTRACT —————
def extraer_csv(ruta: Path) -> pd.DataFrame:
    df = pd.read_csv(ruta)
    print(f" [CSV] {len(df):,} filas | columnas: {list(df.columns)}")
    return df

def extraer_sql(ruta_db: str, tabla: str) -> pd.DataFrame:
    conn = sqlite3.connect(ruta_db)
    df = pd.read_sql_query(f"SELECT * FROM {tabla}", conn)
    conn.close()
    print(f" [SQLite] {len(df):,} filas | columnas: {list(df.columns)}")
    return df

def extraer_json(ruta: Path) -> pd.DataFrame:
    datos = json.loads(ruta.read_text())
    df = pd.DataFrame.from_dict(datos, orient='index').reset_index()
    df.rename(columns={'index': 'sensor_id'}, inplace=True)
    print(f" [JSON] {len(df)} filas | columnas: {list(df.columns)}")
    return df

print("=== EXTRACT ===")
t0 = time.perf_counter()
df_raw_csv = extraer_csv(RUTA_CSV)
df_raw_sql = extraer_sql('/tmp/averias.db', 'averias')
df_raw_json = extraer_json(Path('/tmp/metadatos.json'))
t_ext = time.perf_counter() - t0
print(f" Tiempo extraccion: {t_ext:.3f} s")

=== EXTRACT ===
[CSV] 1,000 filas | columnas: ['timestamp', 'sensor_id', 'temperatura',
'presion', 'activo']
[SQLite] 200 filas | columnas: ['id', 'sensor_id', 'gravedad', 'fecha']
[JSON] 4 filas | columnas: ['sensor_id', 'planta', 'fabricante',
'instalado']
Tiempo extraccion: 0.007 s

```

```

In [5]: # — Etapa TRANSFORM —————
def transformar_sensores(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()

```

```

# 1. Tipos de datos
df['timestamp'] = pd.to_datetime(df['timestamp'])
df['activo'] = df['activo'].map({'True': True, 'False': False, True:

# 2. Eliminar outliers de temperatura (> 200 grados es imposible)
n_outliers = (df['temperatura'] > 200).sum()
df.loc[df['temperatura'] > 200, 'temperatura'] = np.nan
print(f" Outliers eliminados: {n_outliers}")

# 3. Imputar NaN con la mediana del sensor
for sid in df['sensor_id'].unique():
    mask = (df['sensor_id'] == sid) & df['temperatura'].isna()
    mediana = df.loc[df['sensor_id'] == sid, 'temperatura'].median()
    df.loc[mask, 'temperatura'] = mediana

# 4. Feature engineering
df['hora'] = df['timestamp'].dt.hour
df['dia_semana'] = df['timestamp'].dt.dayofweek
df['turno'] = pd.cut(df['hora'], bins=[-1, 7, 15, 23],
                    labels=['noche', 'manana', 'tarde'])
df['temp_norm'] = (df['temperatura'] - df['temperatura'].mean()) / df

# 5. Flag de anomalia: temperatura > media_sensor + 2*std_sensor
df['temp_media_s'] = df.groupby('sensor_id')['temperatura'].transform
df['temp_std_s'] = df.groupby('sensor_id')['temperatura'].transform
df['anomalia'] = (df['temperatura'] > df['temp_media_s'] + 2 * df
df.drop(columns=['temp_media_s', 'temp_std_s'], inplace=True)

print(f" Shape final: {df.shape}")
print(f" Nulos restantes: {df.isnull().sum().sum()}")
return df

def transformar_averias(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    df['fecha'] = pd.to_datetime(df['fecha'])
    df['gravedad_num'] = df['gravedad'].map({'baja': 1, 'media': 2, 'alta

return df

print("=== TRANSFORM ===")
t0 = time.perf_counter()
df_sensores_t = transformar_sensores(df_raw_csv)
df_averias_t = transformar_averias(df_raw_sql)
t_tr = time.perf_counter() - t0
print(f" Tiempo transformacion: {t_tr:.3f} s")
print()
print("Muestra de df_sensores_t:")
print(df_sensores_t[['timestamp', 'sensor_id', 'temperatura', 'temp_norm'

```

=== TRANSFORM ===

```

Outliers eliminados: 5
Shape final: (1000, 10)
Nulos restantes: 51
Tiempo transformacion: 0.023 s

```

Muestra de df_sensores_t:

	timestamp	sensor_id	temperatura	temp_norm	turno
0	2024-01-01 00:00:00	S01	53.17	-1.020797	noche
1	2024-01-01 00:10:00	S04	47.34	-1.531561	noche
2	2024-01-01 00:20:00	S02	84.78	1.748542	noche
3	2024-01-01 00:30:00	S01	66.97	0.188215	noche

```
In [6]: # — Join de fuentes y etapa LOAD —
def integrar_fuentes(df_sens, df_aver, df_meta) -> pd.DataFrame:
    # Agregar averias por sensor: total y max gravedad
    aver_agg = df_aver.groupby('sensor_id').agg(
        n_averias=('id', 'count'),
        max_gravedad=('gravedad_num', 'max')
    ).reset_index()

    # JOIN sensores + averias
    df = df_sens.merge(aver_agg, on='sensor_id', how='left')

    # JOIN con metadatos
    df = df.merge(df_meta, on='sensor_id', how='left')

    # Rellenar NaN de averias (sensores sin averias)
    df['n_averias'] = df['n_averias'].fillna(0).astype(int)
    df['max_gravedad'] = df['max_gravedad'].fillna(0).astype(int)

    return df

def cargar_sqlite(df: pd.DataFrame, ruta_db: str, tabla: str):
    conn = sqlite3.connect(ruta_db)
    df.to_sql(tabla, conn, if_exists='replace', index=False)
    conn.close()

print("=== LOAD ===")
t0 = time.perf_counter()
df_final = integrar_fuentes(df_sensores_t, df_averias_t, df_raw_json)
cargar_sqlite(df_final, '/tmp/dataset_ml.db', 'features_ml')
t_load = time.perf_counter() - t0

print(f" Shape dataset final: {df_final.shape}")
print(f" Columnas: {list(df_final.columns)}")
print(f" Tiempo carga: {t_load:.3f} s")
print()
print(f" Tiempo total del pipeline: {t_ext + t_tr + t_load:.3f} s")

=== LOAD ===
Shape dataset final: (1000, 15)
Columnas: ['timestamp', 'sensor_id', 'temperatura', 'presion', 'activo', 'h
'dia_semana', 'turno', 'temp_norm', 'anomalia', 'n_averias', 'max_gravedad',
'planta', 'fabricante', 'instalado']
Tiempo carga: 0.046 s

Tiempo total del pipeline: 0.076 s
```

4.3 Transformaciones: limpieza, normalización y feature engineering

Las transformaciones son la parte más crítica del pipeline: de su calidad depende directamente calidad del modelo ML. Podemos agruparlas en tres categorías:

Categoría	Ejemplos
Limpieza	Eliminar duplicados, imputar nulos, corregir tipos, filtrar outliers

Categoría	Ejemplos
Normalización	Min-Max, Z-score, logarítmica; encodings de categorías
Feature engineering	Ventanas temporales, interacciones, extracción de fecha/hora

Regla de oro

Transformar siempre dentro del pipeline, nunca en el origen. Las transformaciones deben ser reproducibles, versionadas y revertibles.

Vamos a aplicar un conjunto completo de transformaciones sobre el dataset anterior.

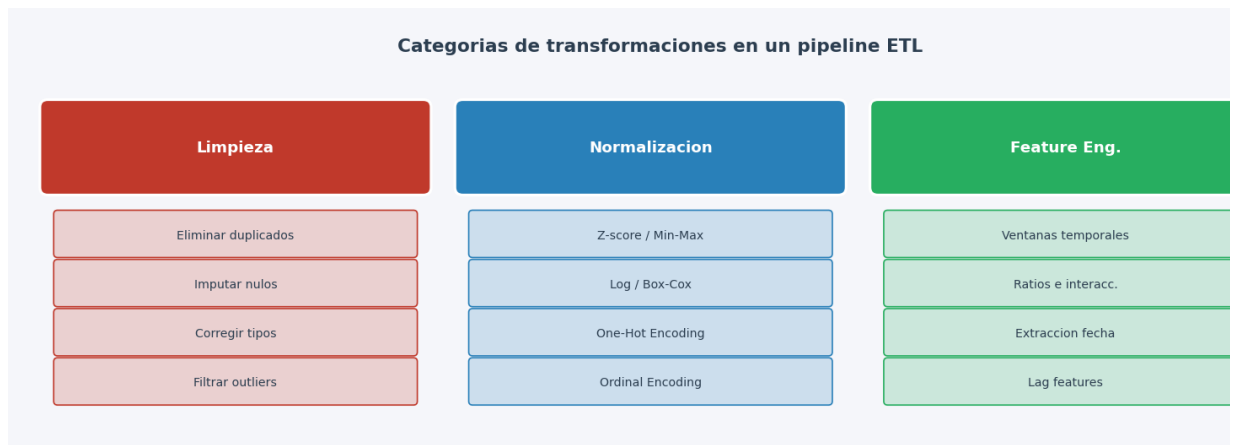


Diagrama 2: Categorías de transformaciones

```
In [8]: # — Transformaciones avanzadas sobre el dataset —
df = df_final.copy()

# 1. Codificacion de categorias
from sklearn.preprocessing import LabelEncoder, MinMaxScaler

le = LabelEncoder()
df['sensor_id_enc'] = le.fit_transform(df['sensor_id'])
df['turno_enc'] = le.fit_transform(df['turno'].astype(str))
df['planta_enc'] = le.fit_transform(df['planta'].astype(str))

# 2. Normalizacion Min-Max de variables numericas continuas
scaler = MinMaxScaler()
cols_num = ['temperatura', 'presion', 'temp_norm']
df[['temp_mm', 'pres_mm', 'tnorm_mm']] = scaler.fit_transform(df[cols_num])

# 3. Feature engineering temporal: ventana rodante (rolling)
df = df.sort_values(['sensor_id', 'timestamp']).reset_index(drop=True)
df['temp_media_6h'] = (df.groupby('sensor_id')['temperatura']
                      .transform(lambda x: x.rolling(6, min_periods=1)
                                  .mean()))
df['temp_std_6h'] = (df.groupby('sensor_id')['temperatura']
                    .transform(lambda x: x.rolling(6, min_periods=1)
                                .std()))

# 4. Flag de anomalia: temperatura > media + 2*std del sensor
stats = df.groupby('sensor_id')['temperatura'].agg(['mean', 'std']).reset_index()
df = df.merge(stats, on='sensor_id', how='left', suffixes=('', '_global'))
df['anomalia'] = (df['temperatura'] > df['mean'] + 2 * df['std']).astype(int)

print("Columnas finales del dataset ML:")
cols_show = ['timestamp', 'sensor_id', 'temperatura', 'temp_mm', 'temp_me
```

```

        'turno_enc', 'n_averias', 'anomalia']
print(df[cols_show].head(6).to_string())
print(f"\nShape: {df.shape} | Anomalias detectadas: {df['anomalia'].sum}

```

Columnas finales del dataset ML:

	timestamp	sensor_id	temperatura	temp_mm	temp_media_6h	turno_
n_averias	anomalia					
0	2024-01-01 00:00:00	S01	53.17	0.331367	53.1700	
1	40	0				
1	2024-01-01 00:30:00	S01	66.97	0.516354	60.0700	
1	40	0				
2	2024-01-01 02:00:00	S01	82.60	0.725871	67.5800	
1	40	0				
3	2024-01-01 02:30:00	S01	51.61	0.310456	63.5875	
1	40	0				
4	2024-01-01 02:40:00	S01	74.20	0.613271	65.7100	
1	40	0				
5	2024-01-01 02:50:00	S01	69.28	0.547319	66.3050	
1	40	0				

Shape: (1000, 25) | Anomalias detectadas: 23

```

In [9]: # Visualizacion de la distribucion antes y despues de normalizar
fig, axes = plt.subplots(1, 3, figsize=(13, 4))

ax1, ax2, ax3 = axes

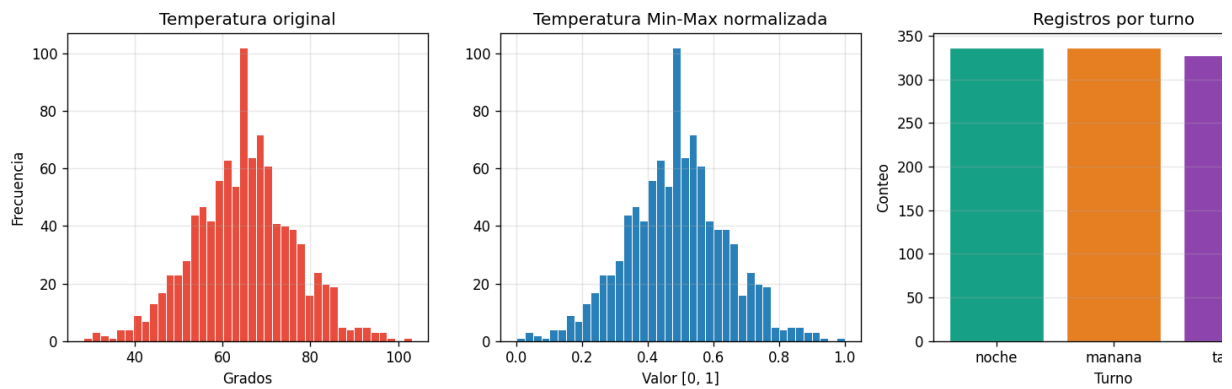
ax1.hist(df_final['temperatura'].dropna(), bins=40, color='#e74c3c',
         edgecolor='white', linewidth=0.6)
ax1.set_title('Temperatura original')
ax1.set_xlabel('Grados')
ax1.set_ylabel('Frecuencia')
ax1.grid(True, alpha=0.3)

ax2.hist(df['temp_mm'].dropna(), bins=40, color='#2980b9',
         edgecolor='white', linewidth=0.6)
ax2.set_title('Temperatura Min-Max normalizada')
ax2.set_xlabel('Valor [0, 1]')
ax2.grid(True, alpha=0.3)

turno_counts = df['turno'].value_counts()
ax3.bar(turno_counts.index, turno_counts.values,
        color=['#16a085', '#e67e22', '#8e44ad'], edgecolor='white', linewidth=1)
ax3.set_title('Registros por turno')
ax3.set_xlabel('Turno')
ax3.set_ylabel('Conteo')
ax3.grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()

```



4.4 Orquestación de pipelines: Airflow y alternativas

Un pipeline ETL que funciona una vez en un cuaderno no es suficiente para producción. Necesitamos **orquestación**: planificar, monitorear y recuperar pipelines automáticamente.

Apache Airflow

El estándar de facto para orquestación de pipelines de datos. Los pipelines se definen como **DA** (Directed Acyclic Graphs) en Python:

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime

dag = DAG('pipeline_sensores',
          schedule_interval='@daily',
          start_date=datetime(2024, 1, 1))

t1 = PythonOperator(task_id='extraer', python_callable=extraer, dag=dag)
t2 = PythonOperator(task_id='transformar', python_callable=transformar, dag=dag)
t3 = PythonOperator(task_id='cargar', python_callable=cargar, dag=dag)

t1 >> t2 >> t3 # dependencias
```

Alternativas a Airflow

Herramienta	Enfoque	Cuándo usar
Prefect	Python nativo, cloud-friendly	Proyectos nuevos sin infraestructura
dbt	Transformaciones SQL declarativas	ELT sobre DW SQL
Luigi	Dependencias simples	Pipelines pequeños sin UI
Dagster	Orientado a assets (datos)	ML platforms modernas
cron + Python	Sencillez máxima	Pipelines simples y estables

El diagrama siguiente muestra la arquitectura de un pipeline orquestado con Airflow.

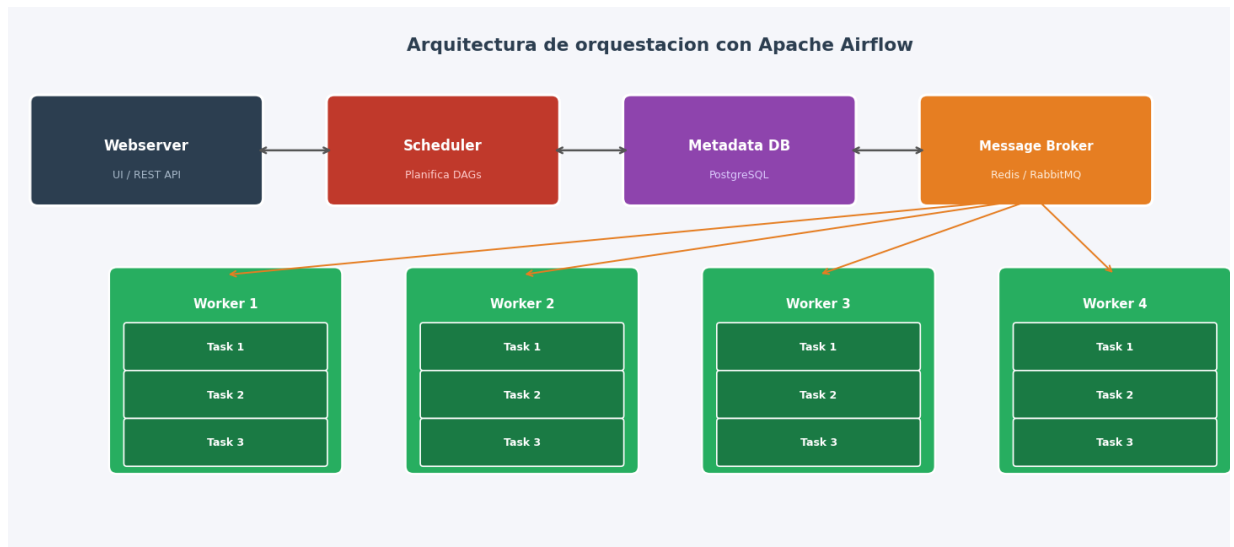


Diagrama 3: Arquitectura de orquestacion Airflow

```

In [11]: # Mini-orquestador: simula la logica de un DAG sin Airflow
# Cada tarea registra su estado, duracion y resultado

class TaskRun:
    def __init__(self, nombre, funcion, *args, **kwargs):
        self.nombre = nombre
        self.funcion = funcion
        self.args = args
        self.kwargs = kwargs
        self.estado = 'pending'
        self.duracion = None
        self.resultado = None
        self.error = None

    def ejecutar(self):
        self.estado = 'running'
        t0 = time.perf_counter()
        try:
            self.resultado = self.funcion(*self.args, **self.kwargs)
            self.estado = 'success'
        except Exception as e:
            self.estado = 'failed'
            self.error = str(e)
        self.duracion = time.perf_counter() - t0
        return self

class DAGRun:
    def __init__(self, nombre):
        self.nombre = nombre
        self.tareas = []
        self.inicio = None
        self.fin = None

    def agregar(self, tarea):
        self.tareas.append(tarea)
        return self

    def ejecutar(self):
        self.inicio = datetime.now()
        print(f"DAG '{self.nombre}' iniciado: {self.inicio:%H:%M:%S}")
        print("-" * 50)

```

```

for t in self.tareas:
    t.ejecutar()
    icono = 'OK' if t.estado == 'success' else 'FAIL'
    print(f" [{icono}] {t.nombre:<20s} {t.duracion:.3f} s")
    if t.estado == 'failed':
        print(f"          Error: {t.error}")
        break
self.fin = datetime.now()
duracion = (self.fin - self.inicio).total_seconds()
estados = [t.estado for t in self.tareas]
ok = estados.count('success')
print("-" * 50)
print(f"Completado en {duracion:.2f} s | {ok}/{len(self.tareas)}")
return self

# Definicion del DAG con nuestras funciones ETL
dag = DAGRun('pipeline_sensores_diario')
dag.agregar(TaskRun('extraer_csv', extraer_csv, RUTA_CSV))
dag.agregar(TaskRun('extraer_sql', extraer_sql, '/tmp/averias.db', 'ave
dag.agregar(TaskRun('extraer_json', extraer_json, Path('/tmp/metadatos.j

dag.ejecutar()

```

DAG 'pipeline_sensores_diario' iniciado: 17:54:30

```

-----
[CSV]      1,000 filas | columnas: ['timestamp', 'sensor_id', 'temperatura'
'presion', 'activo']
[OK] extraer_csv              0.004 s
[SQLite] 200 filas | columnas: ['id', 'sensor_id', 'gravedad', 'fecha']
[OK] extraer_sql              0.001 s
[JSON]    4 filas | columnas: ['sensor_id', 'planta', 'fabricante',
'instalado']
[OK] extraer_json              0.001 s
-----

```

Completado en 0.01 s | 3/3 tareas OK

Out[11]: <__main__.DAGRun at 0x777719f45e40>

4.5 Calidad de datos y monitoreo en producción

Un pipeline ETL de producción no puede confiar ciegamente en los datos de entrada. Necesita **validaciones** que detecten problemas antes de que lleguen al modelo:

Tipos de validaciones

Tipo	Qué verifica	Ejemplo
Esquema	Columnas y tipos correctos	temperatura debe ser float
Rango	Valores dentro de límites	0 <= presion <= 10
Nulos	Porcentaje aceptable de nulos	máx 5% en columna clave
Cardinalidad	Categorías esperadas	sensor_id solo S01–S04
Freshness	Datos recientes	timestamp máx < 24 h
Unicidad	Sin duplicados en clave	(timestamp, sensor_id) único

Herramientas dedicadas

- **Great Expectations**: validaciones declarativas con informes HTML.
- **Pandera**: esquemas de pandas con tipos y restricciones.
- **dbt tests**: validaciones SQL integradas en el pipeline ELT.

Vamos a implementar un validador ligero basado en pandas.

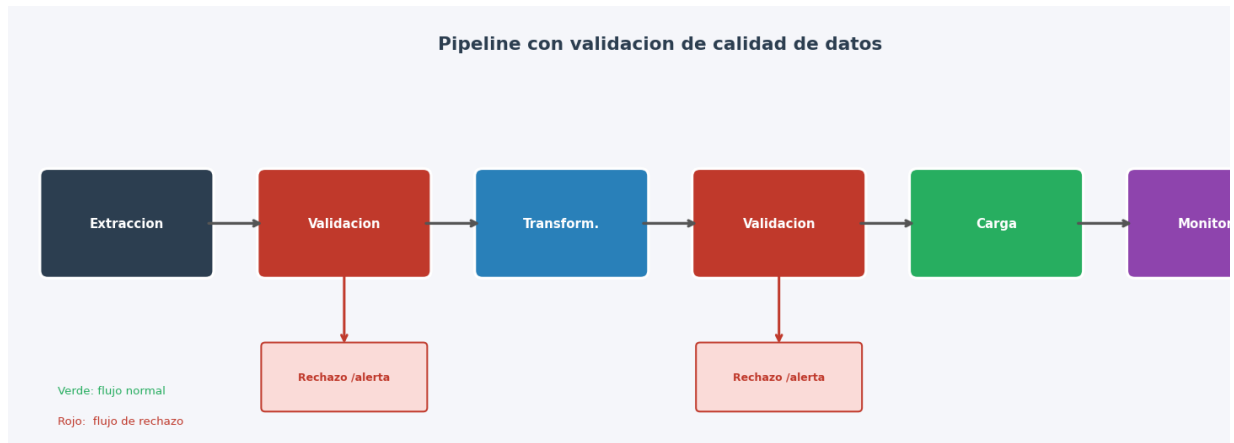


Diagrama 4: Pipeline con validacion de calidad de datos

```
In [13]: # Validador ligero de calidad de datos
class ValidadorDatos:
    def __init__(self, nombre):
        self.nombre = nombre
        self.checks = []
        self.n_ok = 0
        self.n_fail = 0

    def check(self, descripcion, condicion, detalle=''):
        resultado = bool(condicion)
        estado = 'OK' if resultado else 'FAIL'
        self.checks.append({'descripcion': descripcion, 'ok': resultado,
                           'estado': estado, 'detalle': detalle})
        if resultado:
            self.n_ok += 1
        else:
            self.n_fail += 1
        print(f" [{estado}] {descripcion} {detalle}")

    def resumen(self):
        total = self.n_ok + self.n_fail
        print(f" ---")
        print(f" Resultado: {self.n_ok}/{total} checks OK ")
        print(f" ({'PASS' if self.n_fail == 0 else 'FAIL - revisar pipeline'})")
        return self.n_fail == 0

# Validar el dataset transformado
v = ValidadorDatos('dataset_sensores')
print(f"=== Validacion de calidad: {v.nombre} ===")

cols_requeridas = ['timestamp', 'sensor_id', 'temperatura', 'presion', 't
v.check("Columnas requeridas presentes",
        all(c in df.columns for c in cols_requeridas))

v.check("Sin nullos en sensor_id",
```

```

df['sensor_id'].isna().sum() == 0,
f"({df['sensor_id'].isna().sum()} nulos)")

pct_nulos_temp = df['temperatura'].isna().mean() * 100
v.check("Nulos temperatura < 5%",
        pct_nulos_temp < 5.0,
        f"({pct_nulos_temp:.1f}%)")

v.check("Rango temperatura [10, 150]",
        df['temperatura'].dropna().between(10, 150).all(),
        f"(min={df['temperatura'].min():.1f}, max={df['temperatura'].max(

sensores_validos = {'S01', 'S02', 'S03', 'S04'}
sensores_en_datos = set(df['sensor_id'].unique())
v.check("Sensores conocidos",
        sensores_en_datos.issubset(sensores_validos),
        f"({sensores_en_datos})")

n_dup = df.duplicated(subset=['timestamp', 'sensor_id']).sum()
v.check("Sin duplicados (timestamp, sensor_id)",
        n_dup == 0,
        f"({n_dup} duplicados)")

ts_max = pd.to_datetime(df['timestamp']).max()
dias_diff = (datetime.now() - ts_max.to_pydatetime()).days
v.check("Datos recientes (< 365 dias)",
        dias_diff < 365,
        f"({dias_diff} dias desde el ultimo registro)")

ok = v.resumen()
print(f"{'Pipeline VALIDADO' if ok else 'Pipeline CON ALERTAS'}")

```

```

=== Validacion de calidad: dataset_sensores ===
[OK ] Columnas requeridas presentes
[OK ] Sin nulos en sensor_id (0 nulos)
[OK ] Nulos temperatura < 5% (0.0%)
[OK ] Rango temperatura [10, 150] (min=28.4, max=103.0)
[OK ] Sensores conocidos ({'S04', 'S02', 'S03', 'S01'})
[OK ] Sin duplicados (timestamp, sensor_id) (0 duplicados)
[FAIL] Datos recientes (< 365 dias) (949 dias desde el ultimo registro)
---
Resultado: 6/7 checks OK (FAIL – revisar pipeline)
Pipeline CON ALERTAS

```

4.6 Pipeline completo: de fuentes múltiples a dataset ML ready

Vamos a encadenar todo lo visto en un pipeline reproducible que:

1. Extrae datos de tres fuentes heterogéneas
2. Valida la calidad en la entrada
3. Transforma y enriquece
4. Valida la calidad en la salida
5. Carga el dataset final en SQLite y Parquet
6. Genera un informe de ejecución

Este patrón es el que se usa en producción, escalado con Airflow o Prefect.

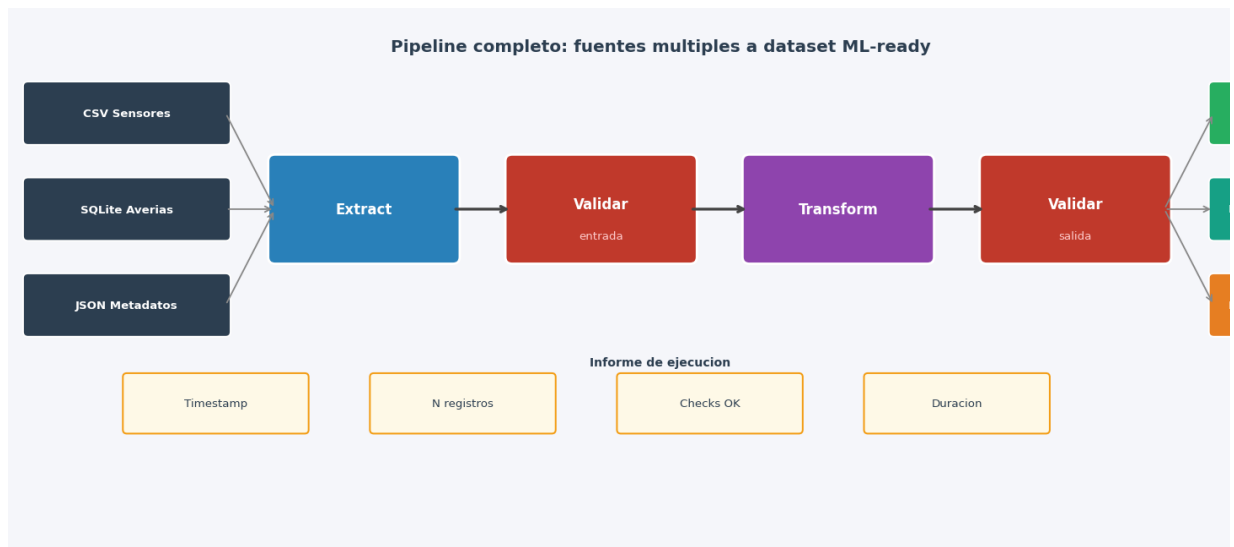


Diagrama 5: Pipeline completo

```
In [15]: # Pipeline completo integrado
def pipeline_completo(cfg: dict) -> dict:
    informe = {
        'inicio':      datetime.now().isoformat(),
        'n_extraidos': 0,
        'n_finales':   0,
        'checks_ok':    0,
        'checks_total': 0,
        'errores':      [],
        'duracion_s':   0.0,
    }
    t_total = time.perf_counter()

    # — EXTRACT —
    try:
        df_s = extraer_csv(Path(cfg['csv']))
        df_a = extraer_sql(cfg['db'], 'averias')
        df_m = extraer_json(Path(cfg['json']))
        informe['n_extraidos'] = len(df_s)
    except Exception as e:
        informe['errores'].append(f"EXTRACT: {e}")
        return informe

    # — VALIDACION ENTRADA —
    v_in = ValidadorDatos('entrada')
    v_in.check("CSV no vacio", len(df_s) > 0, f"({len(df_s)} filas)")
    v_in.check("Columna temperatura existe", 'temperatura' in df_s.columns)
    v_in.check("Columna sensor_id existe", 'sensor_id' in df_s.columns)
    informe['checks_ok'] += v_in.n_ok
    informe['checks_total'] += v_in.n_ok + v_in.n_fail

    # — TRANSFORM —
    df_s_t = transformar_sensores(df_s)
    df_a_t = transformar_averias(df_a)
    df_out = integrar_fuentes(df_s_t, df_a_t, df_m)

    # Añadir hash de reproducibilidad por fila
    df_out['row_hash'] = df_out.apply(
        lambda r: hashlib.md5(str(r.to_dict()).encode()).hexdigest()[:10]
```

```

informe['n_finales'] = len(df_out)

# — VALIDACION SALIDA —
v_out = ValidadorDatos('salida')
v_out.check("Sin perdida de filas",
            len(df_out) == informe['n_extraidos'],
            f"({informe['n_extraidos']} -> {len(df_out)})")
v_out.check("Columna row_hash presente", 'row_hash' in df_out.columns)
v_out.check("Sin nulos en sensor_id", df_out['sensor_id'].isna().sum() == 0)
informe['checks_ok'] += v_out.n_ok
informe['checks_total'] += v_out.n_ok + v_out.n_fail

# — LOAD —
# SQLite
cargar_sqlite(df_out, cfg['destino_db'], 'features_ml')
# Parquet
df_out.to_parquet(cfg['destino_parquet'], index=False)

informe['duracion_s'] = round(time.perf_counter() - t_total, 3)
informe['fin'] = datetime.now().isoformat()
return informe

# Ejecutar el pipeline completo
print("=== Pipeline completo ===")
cfg = {
    'csv': '/tmp/sensores.csv',
    'db': '/tmp/averias.db',
    'json': '/tmp/metadatos.json',
    'destino_db': '/tmp/dataset_final.db',
    'destino_parquet': '/tmp/dataset_final.parquet',
}

informe = pipeline_completo(cfg)

print()
print("=== INFORME DE EJECUCION ===")
for k, v in informe.items():
    print(f" {k:<20s}: {v}")

tam_pq = Path(cfg['destino_parquet']).stat().st_size / 1e3
tam_db = Path(cfg['destino_db']).stat().st_size / 1e3
print(f" {'tamano_parquet':<20s}: {tam_pq:.1f} KB")
print(f" {'tamano_sqlite':<20s}: {tam_db:.1f} KB")

```

```

=== Pipeline completo ===
[CSV] 1,000 filas | columnas: ['timestamp', 'sensor_id', 'temperatura', 'presion', 'activo']
[SQLite] 200 filas | columnas: ['id', 'sensor_id', 'gravedad', 'fecha']
[JSON] 4 filas | columnas: ['sensor_id', 'planta', 'fabricante', 'instalado']
[OK ] CSV no vacio (1000 filas)
[OK ] Columna temperatura existe
[OK ] Columna sensor_id existe
Outliers eliminados: 5
Shape final: (1000, 10)
Nulos restantes: 51
[OK ] Sin perdida de filas (1000 -> 1000)
[OK ] Columna row_hash presente
[OK ] Sin nulos en sensor_id

```

```

=== INFORME DE EJECUCION ===
inicio           : 2026-08-14T17:54:31.658827
n_extraidos      : 1000
n_finales        : 1000
checks_ok        : 6
checks_total     : 6
errores          : []
duracion_s       : 0.127
fin              : 2026-08-14T17:54:31.786038
tamano_parquet   : 50.2 KB
tamano_sqlite    : 118.8 KB

```

```

In [16]: # Verificacion del dataset final guardado en Parquet
df_verificacion = pd.read_parquet('/tmp/dataset_final.parquet')
print(f"Dataset ML-ready cargado desde Parquet:")
print(f" Shape:      {df_verificacion.shape}")
print(f" Columnas: {list(df_verificacion.columns)}")
print()
print("Estadisticas descriptivas (numericas):")
print(df_verificacion[['temperatura', 'presion', 'temp_norm',
                        'n_averias', 'anomalia']].describe().round(3).to_

```

Dataset ML-ready cargado desde Parquet:

```

Shape:      (1000, 16)
Columnas: ['timestamp', 'sensor_id', 'temperatura', 'presion', 'activo', 'h
'dia_semana', 'turno', 'temp_norm', 'anomalia', 'n_averias', 'max_gravedad',
'planta', 'fabricante', 'instalado', 'row_hash']

```

Estadisticas descriptivas (numericas):

	temperatura	presion	temp_norm	n_averias	anomalia
count	1000.000	1000.000	1000.000	1000.000	1000.000
mean	64.822	2.490	-0.000	41.267	0.023
std	11.414	0.392	1.000	2.786	0.150
min	28.450	1.253	-3.187	39.000	0.000
25%	57.475	2.211	-0.644	39.000	0.000
50%	64.600	2.496	-0.019	40.000	0.000
75%	71.498	2.750	0.585	46.000	0.000
max	103.050	3.672	3.349	46.000	1.000

Resumen del bloque

Concepto	Clave
ETL	Transforma antes de cargar; protege el destino
ELT	Carga en bruto, transforma in-situ; más ágil con DW modernos
Separación de etapas	Extract / Transform / Load como funciones independientes
Validación de calidad	Antes y después de cada transformación
Orquestación	Airflow / Prefect / cron para planificar y monitorear
Reproducibilidad	Hash de filas, versionado de transformaciones

Ejercicios propuestos

1. **Validador extensible:** Añade al `ValidadorDatos` un check de **freshness** que compruebe el timestamp máximo del CSV es de las últimas 24 horas. ¿Qué devuelve con nuestros datos sintéticos? ¿Por qué?
2. **Pipeline con retry:** Modifica el `DAGRun` para que, si una tarea falla, la reintente hasta N veces antes de marcarla como `failed`. Prueba con una función que falla aleatoriamente el 50 % de las veces.
3. **ETL multiarchivo:** Adapta el pipeline para que lea **todos los CSV** de una carpeta (usa `Path.glob('*.*csv')`) y los concatene antes de transformar. ¿Qué validaciones añadirías para asegurarte de que todos tienen el mismo esquema?
4. **ELT con DuckDB:** Reescribe la etapa de transformación usando DuckDB: carga el CSV en memoria con `duckdb.read_csv()`, aplica las transformaciones como SQL y exporta a Parquet. Crea el código con el enfoque pandas.
5. **Feature store ligero:** Crea una función que guarde el dataset final en SQLite con una columna `version` (timestamp del pipeline) y permita recuperar cualquier versión anterior. ¿Cómo garantizarías que las versiones son inmutables?

Fin del Bloque 4 y de la UD1. La UD2 cubre la extracción desde ficheros planos, JSON, XML y formatos optimizados para ML.